

NET-NEUTRAL AI

Implementation Plan

Day-by-day build roadmap — May 17 through June 10, 2026
GitHub DevDays Hackathon 2026 | Version 1.0 | May 2026

Property	Detail
Pre-build window	May 17 – May 24 (8 days) — learning + repo setup
Build window	May 25 – June 7 (14 days) — feature complete target
Buffer + Polish	June 7 – June 8 (1 day) — bug fixes only
Dry run	June 9 (1 day) — full end-to-end run, fix blockers
Recording day	June 10 — clean demo video recorded and submitted
Hard deadline	June 10, 2026

1. Owner Colour Key

Every day block is colour-coded by primary owner. Tasks may involve collaboration but one person is accountable.

Colour	Owner	Role	Machine
Blue	Janhvesh	CS — ML pipeline + FedAvg	Decent CPU laptop
Purple	IT	Coordinator server + database	Weakest machine — Flask only
Green	ENTC1	Client app + networking	GPU laptop
Amber	ENTC2	Client app + networking	GPU laptop
Dark	All	Whole team task	—
Red	Risk	Buffer / fallback task	—

2. Final Hardware Allocation

Machine	Owner	Specs	Role	Runs
A	IT	i3, 4–8GB RAM	Coordinator	server.py, database.db
B	ENTC1	GPU laptop	Client A + pre-training	client.py, pretrain.py

C	ENTC2	GPU laptop	Client B	client.py
D	Janhvesh	Decent CPU laptop	Client C	client.py

IT fallback plan — activate if IT falls behind by June 1

If IT's coordinator is not functional by June 1, ENTC1 takes over coordinator duties (running Flask on their GPU laptop alongside client A). IT is reassigned to: writing the README, maintaining GitHub Issues, and documenting the database schema. This fallback must be decided on the June 1 GMeet — not later.

3. Phase 0 — Pre-Build (May 17 – May 24)

You are not writing production code yet. This phase has two goals: get Janhvesh's ML fundamentals to a workable level, and get the repo and team structure ready so day 1 of build week is not wasted on setup.

PRE-BUILD LEARNING — Janhvesh only (May 17–22)

Learning priority order — do not skip ahead

1. PyTorch 60-min blitz (2 days) → 2. Build a toy classifier in PyTorch from scratch (1 day) → 3. Read one FedAvg-from-scratch GitHub repo line by line (1 day) → 4. HuggingFace datasets library — load IMDb, inspect it (1 day) → 5. Read model.py spec from TRD and sketch the TransformerClassifier by hand (1 day). Six days of focused learning. Stop at 6 days even if you feel unready — build week teaches faster than tutorials.

Day	Topic	Resource	Done When
May 17	PyTorch tensors + autograd	pytorch.org/tutorials — 60 min blitz Part 1–2	Can create a tensor, run backward(), print gradients
May 18	PyTorch nn.Module + training loop	pytorch.org/tutorials — 60 min blitz Part 3–4	Can write a training loop from scratch with no copy-paste
May 19	Build toy text classifier	Write it yourself: embed → linear → softmax	Model trains on 100 fake samples, loss goes down
May 20	Read FedAvg repo	github.com/AshwinRJ/Federated-Learning-PyTorch	Can explain what each function does in plain English
May 21	HuggingFace datasets	huggingface.co/docs/datasets — load_dataset docs	Can load IMDb, print a sample, check label distribution
May 22	Sketch TransformerClassifier	TRD Section 6.1 — model spec	Have model.py skeleton written with correct layer names

PRE-BUILD SETUP — All team (May 22–24)

May 22–23 — Repo + GitHub Issues setup

Owner	Task	Done When	GitHub Issue
Janhvesh	Create GitHub repo: net-neutral-ai. Set visibility to public.	Repo exists at github.com/[username]/net-neutral-ai	#1
Janhvesh	Commit folder structure from TRD Section 3.1 with empty placeholder files	All folders and empty .py files committed	#2

Janhvesh	Create GitHub Issues for every module — assign to correct teammate	All issues created, assigned, and labelled	#3
Janhvesh	Add shared/config.py with placeholder COORDINATOR_IP	config.py committed — teammates can clone and edit	#4
All	Clone repo on each machine. Activate venv. pip install requirements.	Each machine runs: <code>python -c 'import torch; print(torch.__version__)'</code>	#5

May 24 — GMeet 1 — Team kickoff			
Owner	Task	Done When	GitHub Issue
All	Walk through all 6 documents together — PRD, TRD, App Flow, Schema, Implementation Plan	Every teammate can answer: what does my module do and what does it output	—
All	Confirm hardware setup: IT runs Flask, ENTC1+2 are GPU clients, Janhvesh is CPU client	No ambiguity about who runs what	—
All	Set WhatsApp rule: daily end-of-day message — what you built, what's blocking you	Rule agreed by all members	—
Janhvesh	Share GitHub repo link + Issues with all teammates. Confirm everyone has push access.	All teammates can push to repo	—

4. Phase 1 — Core Build (May 25 – May 31)

Everyone builds their primary module independently. No integration yet — integration starts in Phase 2. The goal of Phase 1 is: each module works correctly in isolation.

WEEK 1 — May 25 to May 31

May 25 — Day 1 — Build starts

Owner	Task	Done When	GitHub Issue
Janhvesh	Write model.py — TransformerClassifier with correct architecture from TRD 6.1	model = TransformerClassifier(); model(dummy_input) runs without error	#6
IT	Write database init — init_db() function from Backend Schema Section 5	python server.py creates database.db with both tables	#7
ENTC1	Write client.py skeleton — reads config.py, prints coordinator IP on start	client.py runs and prints config values without crashing	#8
ENTC2	Write client.py skeleton — same as ENTC1 (copy and adapt)	Same as ENTC1	#9

May 26 — Day 2

Owner	Task	Done When	GitHub Issue
Janhvesh	Write data.py — load IMDb via datasets, tokenise, build DataLoader for one client shard	DataLoader yields batches of (tokens, labels) with correct shapes	#10
IT	Write /register endpoint — POST, adds client_id to list, returns JSON	curl test: POST /register returns {status: ok}	#11
IT	Write /model endpoint — GET, returns checkpoint.pt as binary file download	curl test: GET /model downloads a .pt file	#12
ENTC1+2	Write registration logic in client.py — POST /register, handle response	Client prints 'Registered successfully' on run	#13

May 27 — Day 3

Owner	Task	Done When	GitHub Issue
Janhvesh	Write train.py — local training loop: load model, run 2 epochs, return updated state_dict + metadata	train() returns state_dict, samples_trained, time_seconds	#14
IT	Write /submit endpoint — receives multipart POST, saves weights file, logs credits to SQLite	curl test: POST /submit with dummy file returns {credits, round}	#15
ENTC1+2	Write model download logic — GET /model, torch.load weights into local model	Client downloads weights and loads them without error	#16

May 28 — Day 4

Owner	Task	Done When	GitHub Issue
Janhvesh	Write pretrain.py — trains TransformerClassifier on full 15K dataset for 5 epochs, saves checkpoint.pt	checkpoint.pt exists, accuracy on val set printed at end	#17
IT	Write /status endpoint — returns round_number, clients list, leaderboard from SQLite	GET /status returns correct JSON with leaderboard	#18
ENTC1+2	Write training trigger in client.py — calls train.py after model download	Client trains locally and prints loss after each epoch	#19

May 29 — Day 5

Owner	Task	Done When	GitHub Issue
Janhvesh	Run pretrain.py on ENTC1's GPU — generate actual checkpoint.pt. Verify accuracy.	checkpoint.pt committed to repo. Val accuracy $\geq 60\%$	#20
IT	Write FedAvg logic in fedavg.py — element-wise mean of N state_dicts	Unit test: average 3 identical state_dicts returns same state_dict	#21
ENTC1+2	Write weight submission in client.py — torch.save to temp file, POST /submit multipart	Client prints 'Weights submitted. Credits earned: X'	#22

May 30 — Day 6

Owner	Task	Done When	GitHub Issue
Janhvesh	Write evaluation logic — load global model, run on 2K val set, return accuracy float	evaluate(model) returns a float between 0 and 1	#23
IT	Wire FedAvg into server — after all clients submit, run fedavg.py, evaluate, save new checkpoint	Coordinator prints 'Round X complete. Accuracy: Y%' after FedAvg	#24
ENTC1+2	Write polling loop in client.py — GET /status every 5s, print 'Waiting for round N... (Xs elapsed)'	Client polls correctly and exits polling when new round starts	#25

May 31 — Day 7 — Phase 1 checkpoint

Owner	Task	Done When	GitHub Issue
All	GMeet 2 — each person demos their module working in isolation	Every module runs independently without crashing	—
Janhvesh	model.py + train.py + data.py + evaluate() all working	Full local training loop runs on Janhvesh's machine alone	—
IT	All 4 endpoints working, FedAvg running, credits logging to SQLite	Can run server.py and hit all endpoints via curl	—
ENTC1+2	client.py registers, downloads model, trains, submits, polls	Full client loop runs on ENTC1 machine pointing to a test server	—

Janhvesh	Assess IT progress honestly. If /submit or FedAvg not working — activate fallback plan.	Decision made by end of GMeet, not deferred	—
----------	---	---	---

5. Phase 2 — Integration (June 1 – June 4)

All modules are connected for the first time. Expect breakage. The goal is one complete end-to-end training round by June 2, and five clean rounds by June 4.

WEEK 2 — June 1 to June 4

June 1 — Day 8 — First integration attempt

Owner	Task	Done When	GitHub Issue
IT	Start server.py on IT's machine. Confirm /status reachable from Janhvesh's machine via browser.	http://[IT-IP]:5000/status returns JSON on Janhvesh's browser	#26
Janhvesh	Update config.py with IT's machine IP. Run client.py — attempt full registration + download + train + submit.	One complete client round completes end-to-end	#27
ENTC1+2	Same as Janhvesh — run client.py pointing to IT's coordinator	All 3 clients complete round 1 submissions	#28
IT	Confirm FedAvg runs after 3 submissions. Check SQLite has correct rows.	database.db has 3 credits rows and 1 rounds row after round 1	#29

June 1 is the most important day of the build

If the end-to-end round does not complete on June 1, you have 3 days of buffer before feature-complete deadline. If it does complete, you have 3 days for polish and hardening. Either way — do not panic. Debug systematically: start with /status reachability, then /register, then /model download, then /submit. Fix one endpoint at a time.

June 2 — Day 9 — Multi-round test

Owner	Task	Done When	GitHub Issue
All	Run 5 complete rounds end-to-end. Operator types start_round manually between rounds.	5 rounds complete. Coordinator prints accuracy after each round.	#30
Janhvesh	Verify accuracy improves across 5 rounds. Print round-by-round accuracy table.	Round 5 accuracy > Round 1 accuracy	#31
IT	Verify SQLite has 15 credits rows (3 clients x 5 rounds) and 5 rounds rows	Query: SELECT COUNT(*) FROM credits returns 15	#32
ENTC1+2	Verify terminal output matches format in TRD Section 5.3 exactly	Output format matches spec — no missing fields	#33

June 3 — Day 10 — Error handling + edge cases

Owner	Task	Done When	GitHub Issue
-------	------	-----------	--------------

IT	Test 90-second timeout — kill one client mid-round. Confirm FedAvg runs with 2/3 clients.	Coordinator prints 'client_X timed out' and continues	#34
IT	Test corrupted weight file — send a garbage .pt file. Confirm coordinator skips it gracefully.	Coordinator prints warning, does not crash	#35
ENTC1+2	Test client reconnection — stop and restart client mid-session. Confirm it rejoins next round.	Restarted client receives new round signal and trains normally	#36
Janhvesh	Test evaluation edge case — what if val set accuracy is 0.0 or 1.0? Handle gracefully.	evaluate() never crashes regardless of model state	#37

June 4 — Day 11 — Terminal output polish

Owner	Task	Done When	GitHub Issue
All	GMeet 3 — review all terminal outputs together. Do they look clean on camera?	All outputs match specs in TRD and App Flow documents	—
Janhvesh	Ensure final coordinator summary matches App Flow Section 7 format exactly	Final leaderboard prints correctly after round 5	#38
IT	Add startup banner to server.py — prints system info when coordinator starts	Coordinator prints clean startup message before accepting connections	#39
ENTC1+2	Add client startup banner — prints client_id, coordinator IP, round count on start	Client terminal is immediately readable on camera	#40
All	Close all GitHub Issues that are complete. Open new issues for anything discovered.	GitHub Issues board reflects true state of project	—

6. Phase 3 — Hardening & Polish (June 5 – June 7)

No new features. This phase exists to make what you have bulletproof. The system must run 5 rounds without a single crash or manual intervention beyond the `start_round` command.

FINAL BUILD DAYS — June 5 to June 7

June 5 — Day 12 — Stress test

Owner	Task	Done When	GitHub Issue
All	Run the full 5-round demo 3 times in a row without touching the code between runs	All 3 runs complete cleanly. Zero crashes.	#41
Janhvesh	Check accuracy is consistent across runs — variance should be small	Accuracy at round 5 is within 3% across all 3 test runs	#42
IT	Reset database.db between each run — confirm <code>init_db()</code> recreates tables cleanly	Database resets correctly. No stale data from previous run.	#43
ENTC1+2	Run clients on actual separate machines over WiFi — not localhost simulation	Clients on separate machines complete all 5 rounds successfully	#44

June 6 — Day 13 — README + GitHub Actions

Owner	Task	Done When	GitHub Issue
Janhvesh	Write README.md — project description, architecture diagram description, setup instructions, how to run	README is complete. A stranger can follow it and run the demo.	#45
Janhvesh	Add <code>.github/workflows/lint.yml</code> — runs flake8 on push to main	GitHub Actions badge shows passing on repo page	#46
IT	Write database query reference in README — the 5 queries from Backend Schema Section 4	README has a 'Database' section with all queries	#47
ENTC1+2	Write client setup section in README — Windows venv setup, <code>config.py</code> editing, how to start client	README client section is accurate and complete	#48

June 7 — Day 14 — Feature complete

Owner	Task	Done When	GitHub Issue
All	GMeet 4 — final review. Run the system together one more time.	System runs 5 complete rounds. All outputs are clean.	—
All	Code freeze after this GMeet. No new features, no refactoring.	Last commit before freeze tagged: git tag v1.0-demo	—
Janhvesh	Confirm <code>checkpoint.pt</code> is committed and loadable on all machines via git pull	All machines can git pull and run immediately	#49
All	Decide recording logistics — whose screen, what layout, OBS or Xbox Game Bar	Recording plan agreed. No decisions left for June 9.	—

7. Phase 4 — Recording (June 8 – June 10)

BUFFER DAY — June 8

June 8 — Buffer — bug fixes only			
Owner	Task	Done When	GitHub Issue
All	Fix any bugs discovered during June 7 GMeet. No new features.	Same system as June 7. Nothing new added.	—
Janhvesh	If no bugs: spend this day reading through all 6 documents and ensuring README matches reality	README is accurate to final code	—
All	Do not use this day to add features. If tempted — read the kill switch.	Feature freeze respected	—

DRY RUN DAY — June 9

June 9 — Full dry run — not recorded			
Owner	Task	Done When	GitHub Issue
All	Meet physically or on GMeet. Run the complete pre-recording checklist from App Flow Section 10.	All 11 checklist items pass	—
All	Run the full demo once — 5 rounds, all clients, real WiFi network	System completes without crash or manual intervention	—
All	Review the run. Anything that looked bad on screen gets fixed now.	Final fix list is empty or trivial	—
ENTC1	Set up Windows Terminal 4-pane layout. Screenshot it. Confirm it looks good.	4 terminals visible and readable at 1080p	—
All	If dry run fails badly — do not record today. Fix tonight, record tomorrow morning.	Decision made collectively, not individually	—

RECORDING DAY — June 10

June 10 — Demo video recording + submission			
Owner	Task	Done When	GitHub Issue
All	Run pre-recording checklist one more time — all 11 items	All pass. Do not skip.	—
ENTC1	Start OBS or Xbox Game Bar recording on ENTC1's machine	Recording active before any terminal is touched	—

All	Execute demo script from App Flow Section 8.2 exactly as written	5 rounds complete. Final leaderboard visible on coordinator terminal.	—
ENTC1	Stop recording 5 seconds after final leaderboard prints. Export video.	Clean .mp4 file with no cuts	—
Janhvesh	Watch the recording once before submitting. Confirm everything is visible and readable.	Recording reviewed and approved	—
Janhvesh	Submit: GitHub repo link + README + demo video to hackathon platform	Submission confirmed before deadline	—

8. GitHub Issues Master List

Create all of these on Day 1. Assign each to the correct teammate. Close them as they're completed. This is your project board.

Issue #	Title	Assignee	Phase	Label
#1	Create repo and folder structure	Janhvesh	Phase 0	setup
#2	Commit placeholder files from TRD	Janhvesh	Phase 0	setup
#3	Create and assign all GitHub Issues	Janhvesh	Phase 0	setup
#4	Add shared/config.py	Janhvesh	Phase 0	setup
#5	All machines clone repo and install deps	All	Phase 0	setup
#6	Write model.py — TransformerClassifier	Janhvesh	Phase 1	ml
#7	Write init_db() and database schema	IT	Phase 1	backend
#8	Write client.py skeleton — ENTC1	ENTC1	Phase 1	client
#9	Write client.py skeleton — ENTC2	ENTC2	Phase 1	client
#10	Write data.py — IMDb DataLoader	Janhvesh	Phase 1	ml
#11	Write /register endpoint	IT	Phase 1	backend
#12	Write /model endpoint	IT	Phase 1	backend
#13	Write registration logic in client.py	ENTC1+2	Phase 1	client
#14	Write train.py — local training loop	Janhvesh	Phase 1	ml
#15	Write /submit endpoint	IT	Phase 1	backend
#16	Write model download logic in client.py	ENTC1+2	Phase 1	client
#17	Write pretrain.py	Janhvesh	Phase 1	ml

#18	Write /status endpoint	IT	Phase 1	backend
#19	Wire train.py into client.py	ENTC1+2	Phase 1	client
#20	Run pretrain.py on ENTC1 GPU, commit checkpoint.pt	Janhvesh	Phase 1	ml
#21	Write fedavg.py — element-wise mean	IT	Phase 1	backend
#22	Write weight submission in client.py	ENTC1+2	Phase 1	client
#23	Write evaluate() — val set accuracy	Janhvesh	Phase 1	ml
#24	Wire FedAvg + evaluate into server.py	IT	Phase 1	backend
#25	Write polling loop in client.py	ENTC1+2	Phase 1	client
#26	First integration — /status reachable cross-machine	IT	Phase 2	integration
#27	First full client round end-to-end	Janhvesh	Phase 2	integration
#28	All 3 clients complete round 1	ENTC1+2	Phase 2	integration
#29	Verify SQLite rows after round 1	IT	Phase 2	integration
#30	Run 5 complete rounds end-to-end	All	Phase 2	integration
#31	Verify accuracy improvement across rounds	Janhvesh	Phase 2	integration
#32	Verify SQLite row counts after 5 rounds	IT	Phase 2	integration
#33	Verify terminal output format matches TRD	ENTC1+2	Phase 2	integration
#34	Test 90-second timeout handling	IT	Phase 2	error-handling
#35	Test corrupted weight file handling	IT	Phase 2	error-handling
#36	Test client reconnection mid-session	ENTC1+2	Phase 2	error-handling
#37	Test evaluate() edge cases	Janhvesh	Phase 2	error-handling
#38	Polish coordinator final summary output	Janhvesh	Phase 2	polish
#39	Add startup banner to server.py	IT	Phase 2	polish
#40	Add startup banner to client.py	ENTC1+2	Phase 2	polish
#41	Stress test — 3 full runs in a row	All	Phase 3	testing
#42	Verify accuracy consistency across runs	Janhvesh	Phase 3	testing
#43	Test database reset between runs	IT	Phase 3	testing
#44	Test on separate machines over WiFi	ENTC1+2	Phase 3	testing

#45	Write README.md	Janhvesh	Phase 3	docs
#46	Add GitHub Actions lint workflow	Janhvesh	Phase 3	ci
#47	Add database query reference to README	IT	Phase 3	docs
#48	Write client setup section in README	ENTC1+2	Phase 3	docs
#49	Commit and verify checkpoint.pt on all machines	Janhvesh	Phase 3	ml

9. Kill Switches

Each of these is a hard decision point. When the condition is met, the action is taken — no debate, no 'let's wait one more day'.

Condition	Date	Action
IT's /submit endpoint not working	June 1	ENTC1 takes over coordinator. IT reassigned to docs + database queries.
FedAvg not converging after 5 rounds	June 2	Switch to 3 epochs per round instead of 2. Rerun pretrain.py.
3-machine WiFi test fails consistently	June 5	Record demo on one machine with 3 simulated clients (localhost). Disclose in README.
Accuracy never improves across rounds	June 5	Check data split — likely data leak. Resplit dataset with strict index ranges.
System crashes during June 9 dry run	June 9	Record a screen capture of the June 5 stress test run as backup video. Submit that if needed.
Demo video quality is poor (unreadable terminals)	June 10	Re-record immediately. Font size 16+. 1080p minimum. Do not submit a blurry video.

10. Daily WhatsApp Sync Template

Every team member sends this message at end of day. No exceptions. If someone doesn't send it — Janhvesh follows up.

```
Daily sync — [Name] — [Date]
```

```
DONE TODAY:
```

```
- [Task 1]
```

```
- [Task 2]
```

```
BLOCKED BY:
- [Blocker or 'nothing']

TOMORROW:
- [What I'm building next]

GITHUB:
- Issues closed: #X, #Y
- Issues opened: #Z (if any)
```

Why this matters

A blocker that sits undisclosed for 2 days costs you 2 days. A blocker posted on WhatsApp gets resolved the same evening. The daily sync is not bureaucracy — it is the difference between shipping and not shipping.

11. GMeet Schedule

GMeet	Date	Agenda	Decision Required
GMeet 1	May 24	Walk through all documents. Confirm roles. Set WhatsApp rule.	Hardware allocation confirmed. Repo access confirmed.
GMeet 2	May 31	Each person demos their isolated module.	IT fallback decision made if coordinator not functional.
GMeet 3	June 4	Review all terminal outputs together.	Output format approved for demo. Issues list updated.
GMeet 4	June 7	Final system run together. Code freeze.	Feature freeze confirmed. Recording logistics decided.
GMeet 5	June 9	Dry run review. Fix list.	Go / no-go for recording on June 10.

12. A Note on How This Goes Wrong

The most common failure mode in hackathon teams

Someone falls silent for 3 days, doesn't post their daily sync, and when the GMeet happens it turns out they built nothing and were too embarrassed to say so. By then there is no time to recover. The daily WhatsApp sync exists specifically to make silence impossible. If you see a teammate go quiet for more than one day — call them, not message them.

The second most common failure mode

Integration is left to the last 2 days. Individual modules work perfectly in isolation. When connected, nothing works and there is no time to debug. Phase 2 starts on June 1 — day 8 of 16. That is not too early. That is exactly right.

What success looks like

June 10, all four of you are on a call. The recording plays. 5 rounds complete. Accuracy climbs from 67% to 79%. The leaderboard prints. The video is clean. You submit it with 2 hours to spare. That is the target. Everything in this document exists to make that moment happen.